

# Software Requirements Specification

for

## CodeForge: Rise of Solvers

Version 1.0 approved

Prepared by Group 7 — Athul Kampiyil (2024BCD0037), Harikrishnan S (2024BCS0025), Niranjan R Soorej (2024BCS0117), Mohammed Ashar Husain (2024BCS0137)

IIIT Kottayam

July 26, 2026

## Table of Contents

|                                                     |    |
|-----------------------------------------------------|----|
| Table of Contents .....                             | 2  |
| Revision History.....                               | 3  |
| 1. Introduction .....                               | 4  |
| 1.1 Purpose .....                                   | 4  |
| 1.2 Document Conventions .....                      | 4  |
| 1.3 Intended Audience and Reading Suggestions ..... | 4  |
| 1.4 Product Scope.....                              | 4  |
| 1.5 References.....                                 | 4  |
| 2. Overall Description .....                        | 5  |
| 2.1 Product Perspective .....                       | 5  |
| 2.2 Product Functions.....                          | 5  |
| 2.3 User Classes and Characteristics .....          | 5  |
| 2.4 Use Case Diagram .....                          | 6  |
| 2.5 Use Case List.....                              | 6  |
| 2.6 Operating Environment .....                     | 7  |
| 2.7 Design and Implementation Constraints .....     | 7  |
| 2.8 User Documentation.....                         | 8  |
| 2.9 Assumptions and Dependencies .....              | 8  |
| 3. External Interface Requirements .....            | 8  |
| 3.1 User Interfaces .....                           | 8  |
| 3.2 Hardware Interfaces .....                       | 10 |
| 3.3 Software Interfaces.....                        | 10 |
| 3.4 Communications Interfaces .....                 | 11 |
| 4. Functional Requirements.....                     | 11 |
| 4.1 User Authentication & Account Linking.....      | 11 |
| 4.2 Coding Platform Sync.....                       | 12 |
| 4.3 Personal Code Village Management.....           | 12 |
| 4.4 Async Village Attacks .....                     | 13 |
| 4.5 Guild Management & Territory Control.....       | 13 |
| 4.6 Guild War Room .....                            | 14 |
| 4.7 League & Trophy Progression System .....        | 14 |
| 5. Other Nonfunctional Requirements .....           | 15 |
| 5.1 Performance Requirements .....                  | 15 |
| 5.2 Safety Requirements.....                        | 15 |
| 5.3 Security Requirements.....                      | 15 |
| 5.4 Software Quality Attributes .....               | 16 |
| 5.5 Business Rules .....                            | 16 |
| 6. Other Requirements .....                         | 16 |
| Appendix A: Glossary.....                           | 17 |
| Appendix B: Analysis Models .....                   | 17 |
| Appendix C: To Be Determined List.....              | 18 |

## Revision History

| Name              | Date          | Reason For Changes           | Version |
|-------------------|---------------|------------------------------|---------|
| Niranjan R Soorej | July 26, 2026 | Initial draft of SRS created | 1.0     |

# 1. Introduction

## 1.1 Purpose

This document specifies the software requirements for Version 1.0 of CodeForge: Rise of Solvers, a gamified competitive-programming practice platform. It covers the complete scope of the web application, including the personal Code Village progression system, asynchronous village attacks, guild-based territory control, third-party judge synchronization, and the league and trophy progression system. This SRS is intended to define what the system must do; the corresponding architecture and design artifacts (class, sequence, ER, component, deployment, and software architecture diagrams) will be produced separately during the Architecture Review.

## 1.2 Document Conventions

Requirements are labeled REQ-*<n>* within each functional feature and are numbered independently per feature. Use case identifiers follow the format UC-*<n>*. Each functional requirement is assumed to inherit the priority (High / Medium / Low) stated for its parent feature unless a requirement explicitly states its own priority. Bold text is used for requirement identifiers and key terms; italicized text denotes template guidance or open items still to be resolved.

## 1.3 Intended Audience and Reading Suggestions

This document is intended for the student development team (frontend, backend, and game-visualization developers), the course instructor / project evaluators, and future team members who join the project. Developers should read Sections 3 and 4 closely before implementation; evaluators may focus on Sections 1, 2, and the appendices for scope and traceability; testers should use Section 4 (Functional Requirements) and Section 5 (Nonfunctional Requirements) to derive test cases. Readers unfamiliar with the project should begin with Section 2 (Overall Description) before proceeding to the detailed requirements in Sections 3 through 6.

## 1.4 Product Scope

CodeForge: Rise of Solvers turns individual competitive-programming practice into a strategy game inspired by village-building and clan-war formats. Each solver grows a personal "Code Village" whose buildings represent topics (e.g., Arrays, Graphs, Dynamic Programming) that level up as the solver solves related problems on linked judge accounts. Solvers can launch asynchronous, curated-problem-set "attacks" on other villages, and guilds compete for control of a shared skill map by pooling members' practice scores. The product's goal is to make consistent, deliberate practice measurably more engaging without changing how or where the underlying problems are solved — problem solving itself continues to happen on the linked third-party judges (Codeforces, LeetCode, CodeChef); CodeForge observes, scores, and gamifies that activity.

## 1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Codeforces API documentation — <https://codeforces.com/apiHelp>
- LeetCode and CodeChef public/community API or profile-data references (exact integration method to be finalized; see Appendix C).

- FastAPI documentation — <https://fastapi.tiangolo.com>
- Phaser.js documentation — <https://docs.phaser.io>
- PostgreSQL and Redis official documentation.
- IEEE SRS Template (institution-provided), used as the structural basis for this document.

## 2. Overall Description

### 2.1 Product Perspective

CodeForge: Rise of Solvers is a new, self-contained web product. It is not a replacement for any existing system, but it depends on and layers gamification over external, third-party competitive-programming judges (Codeforces, LeetCode, CodeChef), which remain the systems of record for problem content and submission verdicts. CodeForge periodically synchronizes with these judges to observe each solver's solved-problem history and derive in-app progression; it does not host problems or accept code submissions itself.

### 2.2 Product Functions

At a high level, the product must let a user:

- Create an account and link one or more supported judge profiles (Codeforces, LeetCode, CodeChef).
- Automatically sync solved-problem activity from linked judges and have it reflected as growth in a personal Code Village, where each topic is represented by a structure that levels up as related problems are solved.
- Launch asynchronous attacks on other solvers' villages using problem sets curated from the target's weak topics, and have their own village be defended based on its topic strength.
- Create or join a guild, and contribute individual practice scores toward the guild's control of zones on a shared territory-control map.
- View a guild War Room summarizing member topic strengths to plan coordinated guild-war strategy.
- Progress through a league and trophy system (Bronze through Legend) based on sustained practice and battle performance.
- View personal and guild standings on leaderboards.

### 2.3 User Classes and Characteristics

| User Class                                | Characteristics                                                                                                                                                                                                                                                        |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Solver (Student / Competitive Programmer) | Primary end user. Links judge account(s), grows a personal village, launches and receives attacks, and participates in a guild. Ranges from casual practicers to serious competitive programmers; assumed to have basic familiarity with at least one supported judge. |
| Guild Leader / Officer                    | A Solver with elevated permissions within a guild. Creates or administers a guild, reviews the War Room, and coordinates territory-control strategy on behalf of the guild.                                                                                            |

| User Class        | Characteristics                                                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Guild Member      | A Solver who has joined a guild and whose individual practice score contributes to guild territory control, but who does not hold administrative guild permissions.       |
| Admin / Moderator | Manages user and guild accounts, resolves disputes (e.g., suspected fraudulent sync data), and configures global game-balance settings such as attack cooldown durations. |
| Guest / Visitor   | An unauthenticated visitor who may view public leaderboards and marketing content but cannot access village, attack, or guild features until registered.                  |

## 2.4 Use Case Diagram

The diagram below shows the primary actors (Solver, Guild Leader/Officer, Admin/Moderator, and the external Coding Judge system) and the major use cases they participate in.

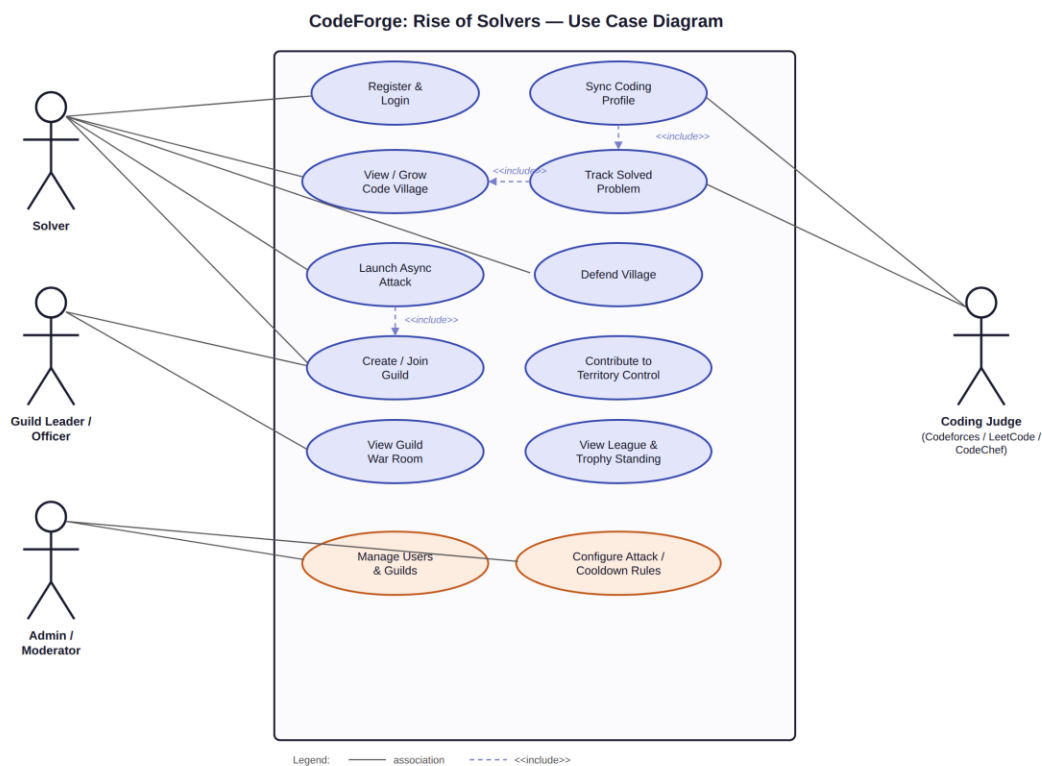


Figure 2.1 — Overall Use Case Diagram for CodeForge: Rise of Solvers

## 2.5 Use Case List

| UC ID | Use Case | Primary Actor |
|-------|----------|---------------|
|-------|----------|---------------|

| UC ID | Use Case                          | Primary Actor                  |
|-------|-----------------------------------|--------------------------------|
| UC-1  | Register & Login                  | Solver                         |
| UC-2  | Sync Coding Profile               | Solver                         |
| UC-3  | View / Grow Code Village          | Solver                         |
| UC-4  | Track Solved Problem              | Coding Judge (external system) |
| UC-5  | Launch Async Attack               | Solver                         |
| UC-6  | Defend Village                    | Solver                         |
| UC-7  | Create / Join Guild               | Solver                         |
| UC-8  | Contribute to Territory Control   | Guild Member                   |
| UC-9  | View Guild War Room               | Guild Leader / Officer         |
| UC-10 | View League & Trophy Standing     | Solver                         |
| UC-11 | Manage Users & Guilds             | Admin / Moderator              |
| UC-12 | Configure Attack / Cooldown Rules | Admin / Moderator              |

## 2.6 Operating Environment

CodeForge is delivered as a responsive web application accessed through modern desktop and mobile browsers (Chrome, Firefox, Edge, Safari — current and previous major versions). The frontend is built with React and uses Phaser.js for the interactive village and territory-map canvases. The backend runs on FastAPI (Python) on Linux-based server infrastructure, backed by a PostgreSQL database for persistent data and Redis for matchmaking, caching, and cooldown-timer state.

## 2.7 Design and Implementation Constraints

- The system depends on external, third-party judges (Codeforces, LeetCode, CodeChef) for problem-solving data; sync accuracy and timeliness are bounded by each judge's API availability, rate limits, and terms of service.
- Codeforces provides an official public REST API; LeetCode and CodeChef do not currently provide equivalent official public APIs, so the sync mechanism for these platforms may depend on community-maintained APIs or profile scraping, which is a known risk (see Appendix C).
- The technology stack is fixed for this project: FastAPI, PostgreSQL, React with Phaser.js, and Redis.
- Near-real-time progress sync must be achieved through polling and/or webhook-based mechanisms, since not all judges support push notifications.
- Attack matchmaking and cooldown enforcement must be implemented using Redis-backed timers to keep them consistent and low-latency across concurrent users.
- Account linking to an external judge profile must be verifiable (e.g., via a one-time verification submission or profile token) to prevent a user from claiming another person's solve history.

## 2.8 User Documentation

The product will be delivered with an in-app onboarding walkthrough (first-time village setup and judge linking), contextual tooltips on the village, attack, and War Room screens, and a guild handbook covering territory-control and war mechanics. An FAQ / help center page will be available from the main navigation.

## 2.9 Assumptions and Dependencies

- It is assumed that solvers have an existing account on at least one supported judge (Codeforces, LeetCode, or CodeChef) before using CodeForge.
- It is assumed that the Codeforces public API remains available and stable throughout development and operation.
- It is assumed that a workable method for retrieving LeetCode and CodeChef solve history (official or community API, or scraping) will remain accessible; if this assumption fails, sync for that judge may be delayed to a later release.
- The project depends on Redis remaining available for real-time matchmaking and cooldown-timer accuracy.
- It is assumed users have a stable internet connection for near-real-time village and territory-map updates.

## 3. External Interface Requirements

### 3.1 User Interfaces

The system provides a browser-based graphical interface built around several major screens, including a guild War Map with territory-control view, a guild roster, season standings, and world events. Representative screens for these major views follow.

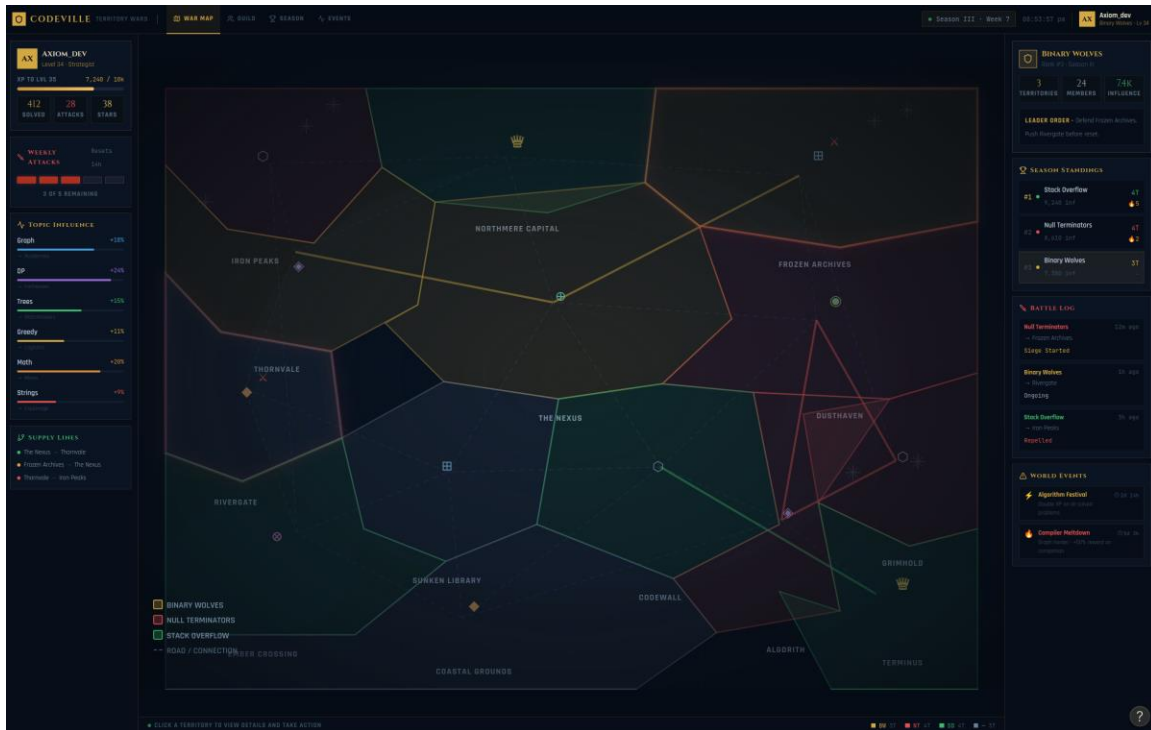


Figure 3.1 — War Map: Territory Control View

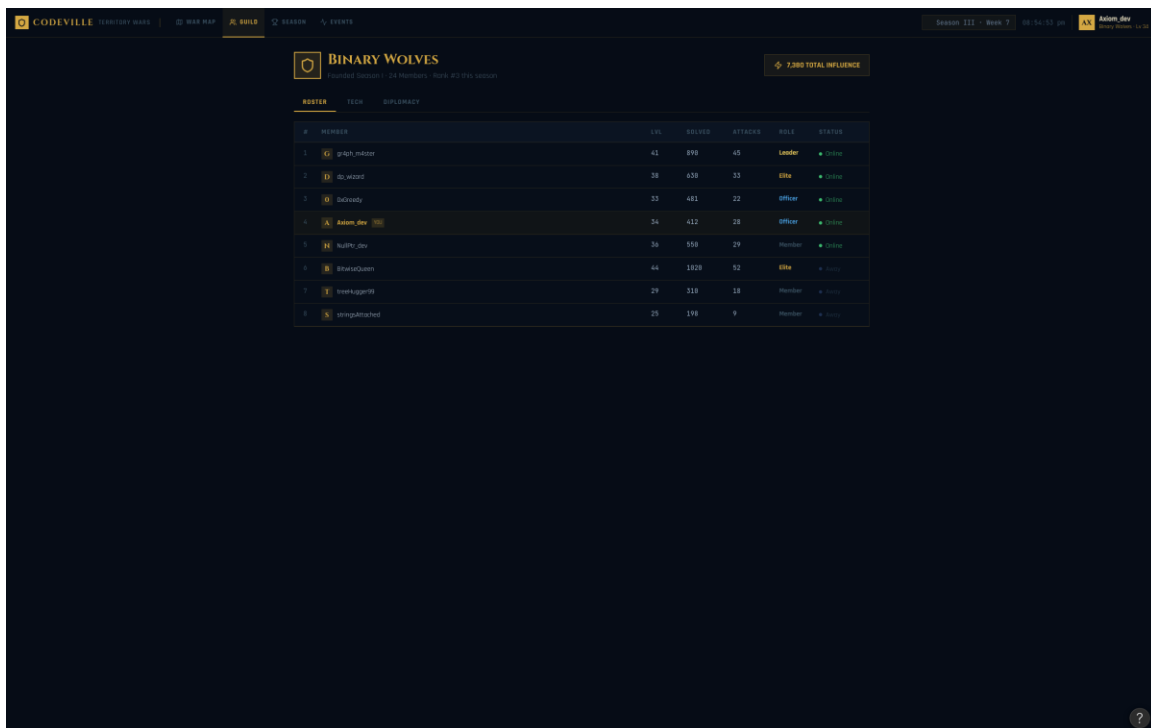


Figure 3.2 — Guild Roster View

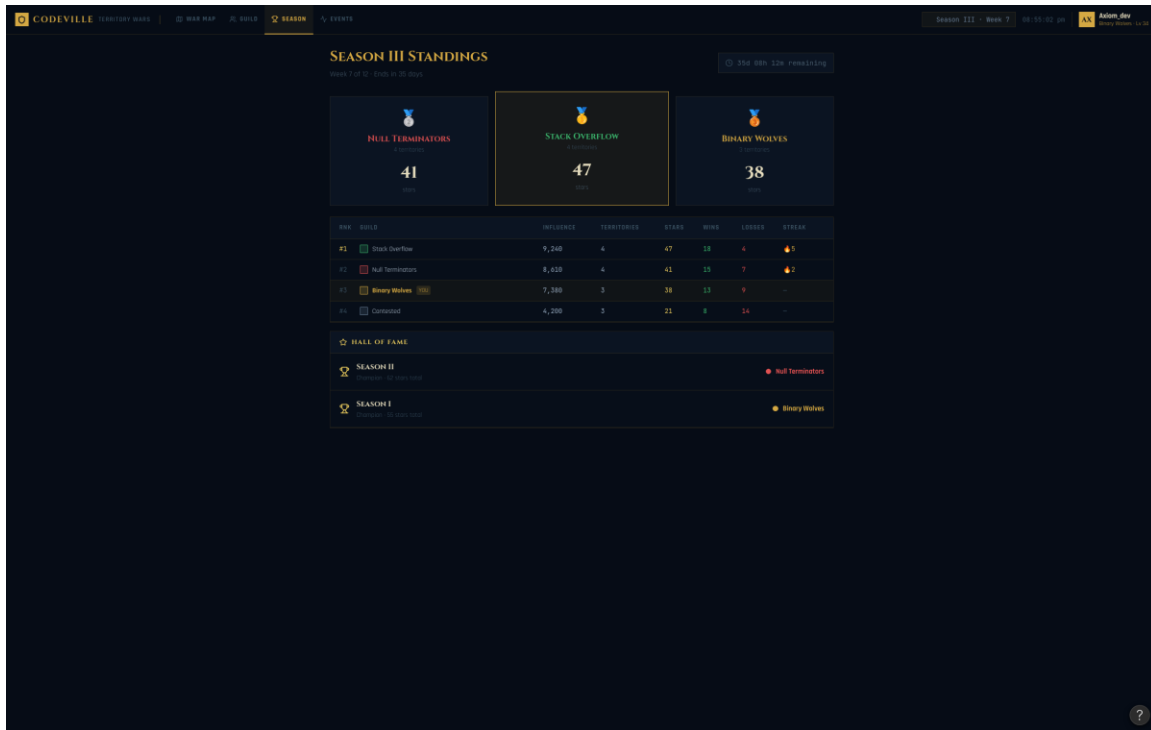


Figure 3.3 — Season Standings View

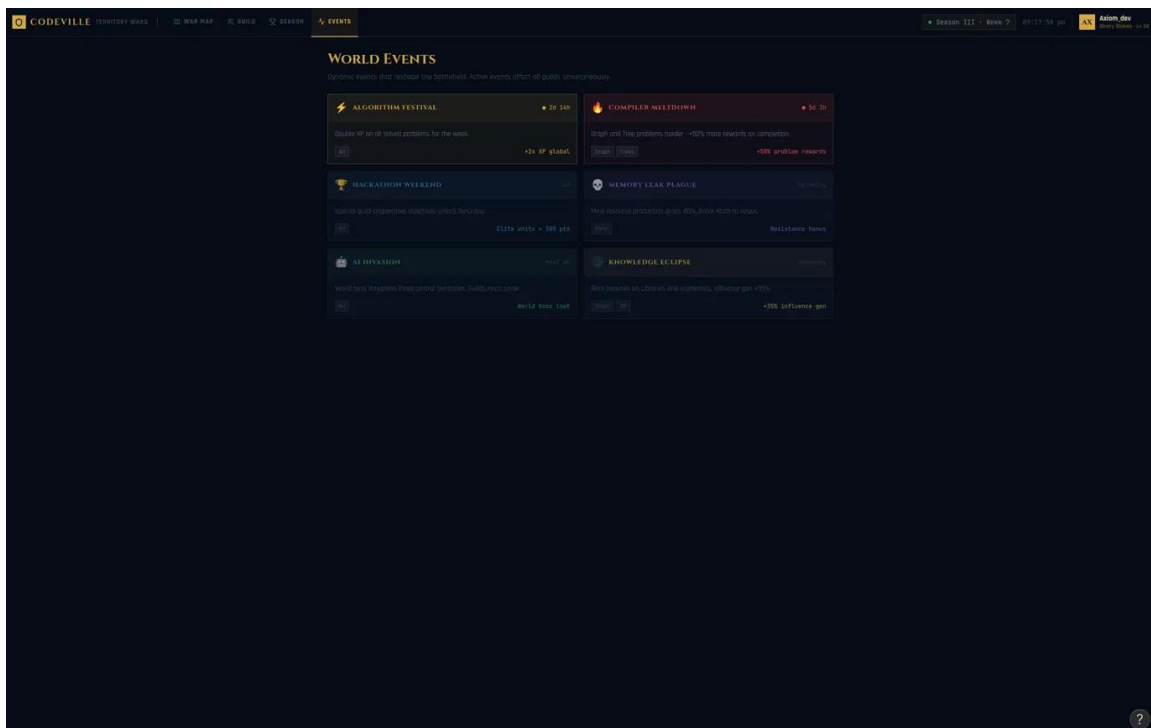


Figure 3.4 — World Events View

### 3.2 Hardware Interfaces

No dedicated or special-purpose hardware is required. The client side runs on standard end-user devices (desktop, laptop, or mobile) with a modern web browser and a network connection. The server side runs on standard cloud or on-premises virtual machines; no direct hardware-level interfacing is required beyond typical server compute, storage, and networking resources.

### 3.3 Software Interfaces

| Software Component                                  | Purpose                                                                                                                                                                                       |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FastAPI backend (REST / WebSocket API)              | Serves all frontend requests, exposes REST endpoints for village, attack, guild, and leaderboard data, and pushes real-time events (e.g., incoming attacks, war-room updates) over WebSocket. |
| PostgreSQL database                                 | Persistent storage for user accounts, village/topic state, guild membership, territory-control history, and league/trophy records.                                                            |
| Redis                                               | Matchmaking queue for attack targeting, attack-cooldown timers, and short-lived caching of frequently-read data (e.g., leaderboards, territory map state).                                    |
| Codeforces API                                      | External source of solved-problem and rating data for solvers who link a Codeforces profile.                                                                                                  |
| LeetCode data source (official or community API)    | External source of solved-problem data for solvers who link a LeetCode profile.                                                                                                               |
| CodeChef data source (official API or profile data) | External source of solved-problem and rating data for solvers who link a CodeChef profile.                                                                                                    |
| React + Phaser.js frontend                          | Renders the interactive UI, including the Code Village and territory-control map canvases.                                                                                                    |

### 3.4 Communications Interfaces

- All client–server communication uses HTTPS, with JSON as the primary data-interchange format for REST requests and responses.
- Real-time in-app events (e.g., an incoming attack notification, a live territory-control change, or a War Room update) are delivered over a WebSocket connection between the client and the FastAPI backend.
- Judge synchronization occurs via scheduled polling of each judge's data source and, where supported, webhook-based notifications; sync jobs run on a periodic schedule (exact interval to be finalized — see Appendix C).
- Any credentials or tokens used to authenticate against linked judge accounts are transmitted and stored using industry-standard encryption in transit and at rest.

## 4. Functional Requirements

Functional requirements are organized by system feature. Each feature corresponds to one or more use cases shown in the Use Case Diagram (Section 2.4).

## 4.1 User Authentication & Account Linking

### 4.1.1 Description and Priority

Allows a person to create a CodeForge account, log in, and link one or more supported judge profiles (Codeforces, LeetCode, CodeChef) so that their solved-problem activity can be tracked. Priority: High.

### 4.1.2 Stimulus/Response Sequences

- User submits registration details → system creates an account and prompts for judge linking.
- User selects a judge and provides a profile handle → system issues a one-time verification step and confirms the link once verified.
- User enters credentials on the login screen → system authenticates and opens the Village Dashboard.

### 4.1.3 Functional Requirements

**REQ-1.1:** The system shall allow a new user to register with a unique email/username and password.

**REQ-1.2:** The system shall allow a registered user to log in and shall reject invalid credentials with a clear error message.

**REQ-1.3:** The system shall allow a user to link at least one supported judge profile (Codeforces, LeetCode, or CodeChef) to their account.

**REQ-1.4:** The system shall verify ownership of a linked judge profile before treating it as an authoritative source of solve data (e.g., via a one-time verification submission or token).

**REQ-1.5:** The system shall allow a user to link additional judge profiles or unlink an existing one after registration.

**REQ-1.6:** The system shall support session management such that a logged-in user remains authenticated for at least 7 days of inactivity via a persistent session token, and can explicitly log out at any time to invalidate that session.

## 4.2 Coding Platform Sync

### 4.2.1 Description and Priority

Periodically retrieves each linked judge's solved-problem history and rating for a solver and reconciles it into the solver's CodeForge profile. Priority: High.

### 4.2.2 Stimulus/Response Sequences

- Scheduled sync job runs → system fetches new solve activity from each linked judge since the last successful sync.
- User manually requests a refresh from the dashboard → system attempts an on-demand sync and reports success, partial success, or failure.
- A judge's data source is unreachable or rate-limited → system retries according to a backoff policy and surfaces a sync-status indicator to the user.

### 4.2.3 Functional Requirements

**REQ-2.1:** The system shall automatically sync each linked judge account on a recurring schedule without requiring user action.

**REQ-2.2:** The system shall allow a user to manually trigger an on-demand sync, limited to once per 5 minutes per linked judge account, to protect judge APIs from excessive request volume.

**REQ-2.3:** The system shall map each newly-solved problem to one or more topic tags used by the Code Village feature.

**REQ-2.4:** The system shall display the last-successful-sync timestamp and current sync status (up to date, in progress, failed) for each linked judge.

**REQ-2.5:** The system shall handle a temporarily unavailable or rate-limited judge data source gracefully, without losing previously-synced data.

## 4.3 Personal Code Village Management

### 4.3.1 Description and Priority

Represents a solver's cumulative topic-wise practice as a personal, visual Code Village, where each topic's structure levels up as related problems are solved. Priority: High.

### 4.3.2 Stimulus/Response Sequences

- A newly-synced solved problem is tagged with one or more topics → system increases the corresponding topic's progress and, once a threshold is met, levels up that topic's structure.
- User opens the Village Dashboard → system renders the current state of every topic structure and an overall village defense rating.

### 4.3.3 Functional Requirements

**REQ-3.1:** The system shall maintain a level for each recognized topic in a solver's village, derived from the solver's synced solve history for that topic.

**REQ-3.2:** The system shall visually reflect a topic's level as a distinct village structure state on the village canvas.

**REQ-3.3:** The system shall compute and display an overall village "defense rating" summarizing the solver's aggregate topic strength.

**REQ-3.4:** The system shall update the village view when new sync data changes a topic's level, without requiring a full page reload.

## 4.4 Async Village Attacks

### 4.4.1 Description and Priority

Lets a solver select a target village and receive a curated problem set drawn from that target's weakest topics, as an asynchronous practice challenge. Priority: High.

### 4.4.2 Stimulus/Response Sequences

- User opens the Attack screen → system suggests candidate target villages via matchmaking based on comparable overall strength.
- User selects a target → system generates a curated problem set from the target's weakest topics and starts a scored attack.

- User completes (or abandons) the curated problem set → system scores the attack outcome and applies any resulting territory-control or trophy effects, then starts the user's attack cooldown.

#### **4.4.3 Functional Requirements**

**REQ-4.1:** The system shall suggest attack targets using a matchmaking mechanism that favors villages of comparable overall strength.

**REQ-4.2:** The system shall generate a curated problem set for an attack based on the target village's lowest-leveled topics.

**REQ-4.3:** The system shall track and score the attacker's performance against the curated problem set (e.g., number solved, time taken).

**REQ-4.4:** The system shall enforce a cooldown period between a solver's consecutive attacks, configurable by an Admin.

**REQ-4.5:** The system shall notify the target solver that their village was attacked and summarize the outcome.

### **4.5 Guild Management & Territory Control**

#### **4.5.1 Description and Priority**

Lets solvers form or join guilds and pool their individual practice scores to contest control of zones on a shared, guild-wide territory-control map. Priority: High.

#### **4.5.2 Stimulus/Response Sequences**

- User creates a guild or requests to join an existing one → system establishes or updates guild membership.
- A guild member's practice score changes (e.g., from solving problems or winning attacks) → system recalculates the guild's contribution toward relevant territory-map zones.
- A territory-control recalculation crosses a control threshold → system updates the map's zone ownership.

#### **4.5.3 Functional Requirements**

**REQ-5.1:** The system shall allow a solver to create a new guild, subject to a unique guild name.

**REQ-5.2:** The system shall allow a solver to request to join an existing guild, and allow a Guild Leader/Officer to approve or reject the request.

**REQ-5.3:** The system shall aggregate member practice scores into a guild-level score for each territory-map zone.

**REQ-5.4:** The system shall update zone ownership on the territory-control map when a guild's aggregated score for that zone surpasses competing guilds according to the territory-control rules.

**REQ-5.5:** The system shall display the current territory-control map with zone ownership visible to all authenticated users.

### **4.6 Guild War Room**

#### **4.6.1 Description and Priority**

Gives Guild Leaders and Officers a consolidated view of member topic strengths to plan coordinated guild-war and territory-control strategy. Priority: Medium.

#### **4.6.2 Stimulus/Response Sequences**

- Guild Leader/Officer opens the War Room → system displays each member's topic-strength summary and recent contribution to territory control.
- Guild Leader/Officer reviews a member's profile from the War Room → system links through to that member's detailed village view.

#### **4.6.3 Functional Requirements**

**REQ-6.1:** The system shall display a per-member summary of topic strengths for all members of a guild to that guild's Leader/Officer role.

**REQ-6.2:** The system shall highlight members whose strengths align with zones the guild is currently contesting.

**REQ-6.3:** The system shall allow a Guild Leader/Officer to view a member's recent contribution to guild territory control from within the War Room.

### **4.7 League & Trophy Progression System**

#### **4.7.1 Description and Priority**

Tracks each solver's long-term progression through a league and trophy system (Bronze through Legend) based on sustained practice and attack/defense performance. Priority: Medium.

#### **4.7.2 Stimulus/Response Sequences**

- A scored event occurs (e.g., attack won, attack successfully defended, sync milestone reached) → system adjusts the solver's trophy count.
- A solver's trophy count crosses a league threshold → system promotes or demotes the solver to the corresponding league and notifies them.

#### **4.7.3 Functional Requirements**

**REQ-7.1:** The system shall maintain a trophy count for each solver, adjusted by defined in-app events (attacks, defenses, and practice milestones).

**REQ-7.2:** The system shall define six league tiers (Bronze, Silver, Gold, Platinum, Diamond, Legend) with trophy thresholds separating them.

**REQ-7.3:** The system shall promote or demote a solver's league standing automatically when their trophy count crosses a tier threshold.

**REQ-7.4:** The system shall display a solver's current league, trophy count, and progress toward the next tier.

**REQ-7.5:** The system shall display global and guild-scoped leaderboards ranked by trophy count or league standing.

## 5. Other Nonfunctional Requirements

### 5.1 Performance Requirements

- **NFR-1.1:** The Village Dashboard and territory-control map shall load their initial view within 2 seconds under normal network conditions.
- **NFR-1.2:** Judge sync jobs shall complete processing of a single solver's new activity within 10 seconds of a successful fetch from the judge's data source, so that village state does not lag noticeably behind an on-demand sync request.
- **NFR-1.3:** Attack matchmaking requests shall return candidate targets within 3 seconds under expected concurrent load.
- **NFR-1.4:** Real-time notifications (incoming attack, territory-control change) shall be delivered to an actively connected client within 3 seconds of the triggering event.

### 5.2 Safety Requirements

- **NFR-2.1:** The system shall not permanently lose a solver's synced solve history or village progression due to a single component failure; sync and progression data shall be persisted durably in PostgreSQL.
- **NFR-2.2:** The system shall validate incoming sync data before applying it to a solver's village state, to prevent corrupted or malformed data from producing an inconsistent village or territory-control state.

### 5.3 Security Requirements

- **NFR-3.1:** The system shall require verified ownership of a linked judge profile before that profile's solve history is used to influence game state, to prevent one user from misappropriating another user's practice record.
- **NFR-3.2:** The system shall store passwords using a secure, salted hashing algorithm and shall never store judge account credentials or tokens in plaintext.
- **NFR-3.3:** The system shall enforce role-based access such that only a guild's Leader/Officer can view that guild's full War Room detail and only an Admin/Moderator can access account- and guild-management functions.
- **NFR-3.4:** The system shall apply rate limiting to attack initiation, sync requests, and guild actions to reduce opportunities for abuse or leaderboard manipulation.
- **NFR-3.5:** The system shall log administrative actions (e.g., account suspension, cooldown-rule changes) for auditability.

### 5.4 Software Quality Attributes

- **Availability:** the core village, attack, and guild features should remain available even if a single third-party judge's data source is temporarily unreachable, degrading gracefully rather than failing entirely.
- **Scalability:** the matchmaking, cooldown, and caching layer (Redis) should be designed to support a growing number of concurrent solvers and guilds without a redesign of core game logic.

- Usability: first-time users should be able to complete account creation, judge linking, and reach their first village view within 5 minutes, via a guided onboarding flow of no more than 5 steps.
- Maintainability: functional features (authentication, sync, village, attacks, guilds, war room, league) should be organized as separable modules to ease independent development and testing.
- Extensibility: the judge-integration layer should be designed so that a new judge (beyond Codeforces, LeetCode, and CodeChef) can be added without redesigning the core village, attack, or territory-control logic.

## 5.5 Business Rules

- Only a Guild Leader or Officer may approve join requests, remove members, or view the full guild War Room.
- A solver may belong to at most one guild at a time.
- A solver is subject to a cooldown period between consecutive attacks; the cooldown duration is configurable by an Admin and applies uniformly unless otherwise stated.
- Territory-control zone ownership is awarded to the guild with the highest aggregated contribution score for that zone, recalculated on a periodic or event-driven basis (exact cadence to be finalized — see Appendix C).
- Only an Admin/Moderator may suspend a user or guild account or modify global game-balance settings such as cooldown durations and league thresholds.

## 6. Other Requirements

- Legal/Compliance: use of third-party judge data must comply with each judge's terms of service and applicable API usage policies; any scraping-based integration must be reassessed for compliance before production release.
- Data retention: synced solve-history and account data should be retained for as long as an account remains active, with a defined deletion process upon account closure (exact policy to be finalized).
- Internationalization: Version 1.0 targets English-language users only; internationalization is out of scope for this release.

## Appendix A: Glossary

| Term                                      | Definition                                                                                                                                                        |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code Village                              | A solver's personal, visual representation of topic-wise practice progress, where each topic is shown as a structure that levels up with related solved problems. |
| Guild                                     | A player-formed group of solvers whose combined practice scores contest control of zones on the shared territory-control map.                                     |
| Territory Control / Territory-Control Map | A shared map divided into topic-based zones, each owned by whichever guild holds the highest aggregated contribution score for that zone.                         |
| War Room                                  | A guild-scoped view, visible to Guild Leaders/Officers, summarizing member                                                                                        |

| Term         | Definition                                                                                                                                                         |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              | topic strengths for war and territory-control planning.                                                                                                            |
| League       | A long-term progression tier (Bronze through Legend) reflecting a solver's sustained performance.                                                                  |
| Trophy       | A point unit awarded or deducted based on attack, defense, and practice outcomes, used to determine league standing.                                               |
| Async Attack | A challenge a solver launches against another solver's village, consisting of a curated problem set drawn from the target's weak topics, completed asynchronously. |
| Cooldown     | A configurable waiting period a solver must observe between consecutive attacks.                                                                                   |
| Judge        | An external, third-party competitive-programming platform (Codeforces, LeetCode, or CodeChef) where problems are actually solved and verdicts are issued.          |
| Topic Level  | A numeric indicator of a solver's practice depth in a given topic, derived from synced solve history.                                                              |

## Appendix B: Analysis Models

*Per the assignment's submission guidelines, the Class Diagram, Sequence Diagram, ER Diagram, Component Diagram, Deployment Diagram, and Software Architecture Diagram are not included in this SRS submission. These design artifacts will be developed and submitted separately during the Architecture Review.*

## Appendix C: To Be Determined List

| TBD ID | Open Item                                                                                                                                                                              |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TBD-1  | Exact integration method for LeetCode and CodeChef solve-history sync (official API availability vs. community API vs. scraping), pending finalization during architecture and design. |
| TBD-2  | Precise territory-control scoring formula and recalculation cadence.                                                                                                                   |
| TBD-3  | Precise matchmaking algorithm for suggesting attack targets (weighting of village strength, recency, guild affiliation, etc.).                                                         |
| TBD-4  | Exact attack-cooldown duration(s) and whether they vary by league tier.                                                                                                                |
| TBD-5  | Trophy award/penalty values for each scored event (attack win, successful defense, practice milestone).                                                                                |
| TBD-6  | Maximum guild size and guild-creation eligibility rules.                                                                                                                               |
| TBD-7  | Sync job scheduling interval and on-demand sync rate limits.                                                                                                                           |